COIT11237

LECTURE NOTES

Authored by
Lachlan J. W. Wyer

Document Start Date: 11th of June 2026
Latest Update: June 13, 2026

Contents

1 Introduction to Databases **3**
- 1.1 What is MySQL?: . . . 3
- 1.2 Databases: . . . 3
- 1.3 Problems with Spreadsheets . . . 3
- 1.4 Database System Components: . . . 3
- 1.5 SQL: . . . 4
- 1.6 Personal DBs: . . . 4
- 1.7 Commercial DBs: . . . 4

2 SQL Part 1 **4**
- 2.1 Keys: . . . 4
  - 2.1.1 Primary . . . 4
  - 2.1.2 Composite . . . 4
  - 2.1.3 Foreign . . . 5
- 2.2 SELECT Clause: . . . 5
- 2.3 WHERE Clause: . . . 5
- 2.4 Ordering Rows: . . . 5
- 2.5 Having: . . . 5
- 2.6 Subqueries: . . . 6

3 SQL - Part 2 **6**
- 3.1 Joins of Tables: . . . 6
- 3.2 Union: . . . 6
- 3.3 Modifying Data: . . . 6

4 ER Diagrams **7**
- 4.1 Purpose: . . . 7
- 4.2 Database Definition: . . . 7
- 4.3 Entity Representation: . . . 7
- 4.4 Identifier: . . . 7
- 4.5 Relationships: . . . 7
- 4.6 Modelling ERDs: . . . 8
- 4.7 Weak Entity: . . . 8
  - 4.7.1 Id-dependent . . . 8
  - 4.7.2 Non-Identifying . . . 9

5 ER Digrams - 2 **9**
5.1 Associative Entities: 9
5.2 Subtype: 9
5.3 Notation: 9
5.4 Recursive Relationships: 10

6 Normalization **10**
6.1 Data Redundancy: 10
6.2 Relational DBs: 10
6.3 Relational Checklists: 11
6.4 Candidate Keys: 11
6.5 Primary Keys: 11
6.6 Boyce-Codd Normal Form: 11
6.6.1 Referential Integrity: 11
6.6.2 NULL Values: 11
6.6.3 Functional Dependancy: 12
6.7 BCNF: 12

7 Database Design **12**
7.1 ER Diagrams to Tables: 12
7.2 Constraints: 12
7.2.1 Checks 12
7.3 Referential Integrity: 13
7.4 Overall Steps: 13

1 Introduction to Databases

1.1 What is MySQL?:

Structured Query Language (SQL) is the language used to manage relational database management systems (RDBMS). It follows the CRUD principles. In that it Create, Replace, Update, & and Deletes. SQL is used in both small & large scale applications.

1.2 Databases:

A database (DB) can be made up of multiple tables. The conventional naming is singular. For examples, a customers' information DB would just be named customer, not customers. The table also needs a primary key (PK) as a unique identifier. This is so a singular record can be found. Tables can also be linked together through foreign keys (FK) which is just a PK in another table. When this is done, a relational database is created.

1.3 Problems with Spreadsheets

1. Have redundant/wrong/out-of-date data present within the sheet. This can create conflicting copies throughout.
2. Multiple themes per row creates modification problems
    - Insertion anomalies
    - Deletion anomalies

1.4 Database System Components:

- Users use forms/reports to modify data
- Database application converts user inputs into SQL data, which is then sent to DBMS
- DBMS:
    - Processes the SQL
    - Enforces the DB rules
    - Provides concurrency, security, back-up and recovery
- Database is where the data is physically stored

1.5 SQL:

SQL is an international standard for the talking with DBs and their tables. For example:

```
SELECT *
FROM Table.name
WHERE ID = 1
```

1.6 Personal DBs:

These are typically just local DBs and tables stored on the system. They can be found with the localhost flags. A crucial part of local DBs is that they only support one connection at a time.

1.7 Commercial DBs:

Multiple connections with many things happening at once. Connections are usually facillitated through JAVA or C#.

2 SQL Part 1

2.1 Keys:

2.1.1 Primary

When creating a table in SQL it must have a unique identifier. For example:

EmployeeID	FirstName
1	Lachlan
2	Paul
3	Paul

2.1.2 Composite

Sometimes a comp PK is utilized to define N:M relational tables. As long as the pair of keys is unique, the table will not break. For example:

ProjectID	EmployeeID
1200	1
1300	1
1300	2
1400	2

2.1.3 Foreign

These keys allow relationships between tables to form. The link must be done back to a PK in the other table. When setting a FK during the creation of the table there are two points to understand:

1. Cascade Delete: Once the original PK is deleted all rows with it as a FK
2. Cascade Update: Same as delete, but with updating all FK rows

2.2 SELECT Clause:

Used to select rows of data from a table. There are only two compulsary parts to the statements. That being the column names and table name or names. To remove duplicate rows doing a SELECT DISTINCT statement will remove duplicate rows.

2.3 WHERE Clause:

Filters rows by constraining items that are in the column name. For example:
SELECT EmployeeName
FROM employee
WHERE EmployeeID = 1

Further constraints can be added to the WHERE clause. Such as, IN, NOT, ALL, BETWEEN, IS NULL/IS NOT NULL, and LIKE. The LIKE can be defined with % for infinite number of characters and _ for 1 character. The _ can be done multiple times (e.g. _ _ _, but joined together like ___) for a set defined number of characters.

2.4 Ordering Rows:

Use the ORDER BY after the WHERE. Ordering can either be defined DESC or ASC, with most DBMSs having ASC as the default.

2.5 Having:

This is basically just a WHERE clause for grouped functions. When in a statement with WHERE clauses, the WHERE clause is applied before the HAVING.

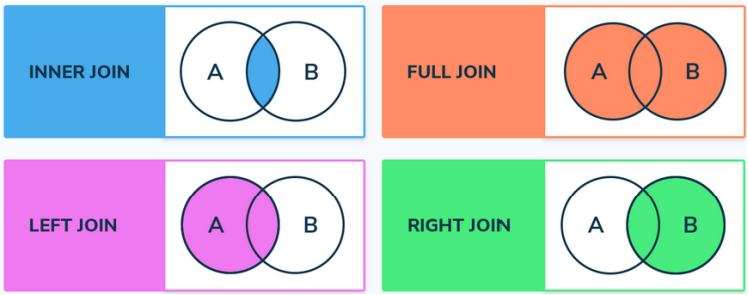
2.6 Subqueries:

SELECT *
FROM table1
WHERE column IN (subquery)

3 SQL - Part 2

3.1 Joins of Tables:

There a number of different types of joins. Joins can be used consecutaley to have multiple different tables joined together. The column used to join the table must be the same in each one. For example, an INNER JOIN is similary to,
FROM table1 as T1
INNER JOIN table2 as T2 ON T1.id = T2.id



3.2 Union:

These are used to completely join two tables together. This is similar to a full join. For example,
(SELECT * FROM table1)
UNION
(SELECT * FROM table2)

3.3 Modifying Data:

Used when updating/deleting a row of data. As well as creating or dropping a table.

4 ER Diagrams

A formal presentation of user requirements. They help refine the requirements before a database is created.

4.1 Purpose:

Help visualizes the relationships between the tables.

- What thing's are stored within the DB?
- What do we want to know about these things?
- Are there any relationships between the tables?
- Are there any constraints in the DB?

To summmarize ER diagrams help define the entire requirements of the DB system.

4.2 Database Definition:

A DB contains a definition of its data. Specifically, the meta-data is the specific data describing the structure of DBs data. These include the datatype (INT, DATE, CHAR & VARCHAR), uniqueness, auto-increment, and length.

4.3 Entity Representation:

Simply, name it an entity if it is more than just a name of something. I.E. Has non-key attributes or relationships with multiple entities.

4.4 Identifier:

Entity instances have an identifier. These are attributes that are unique (E.g. PKs) across all instances. Sometimes a composite of 2 or more identifiers to make a unique key.

4.5 Relationships:

Minimum cardinality: The minimum number of instances that must be involved with an instance.
Maximum cardinality: The maximum number of instances that a relationships can have.

Symbol	Meaning	Numeric Meaning
	Mandatory—One	Exactly one
	Mandatory—Many	One or more
	Optional—One	Zero or one
	Optional—Many	Zero or more

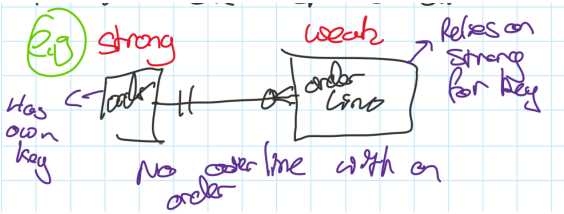
When drawing these, if connected by a dashed line, both entities are independent of each other.

4.6 Modelling ERDs:

1. Write out all entities
2. Display relationships and state any assumptions made with them
3. Check if any extra modelling is required

4.7 Weak Entity:

A WE is an entity that cannot exist in a DB without another entity also existing. That being if the strong entity is removed, the weak one must also be removed.



4.7.1 Id-dependent

This relies on its parent entity for its identifier. They usually have composite identifiers within the DB. Represented through a solid line.

4.7.2 Non-Identifying

Does not rely on its parent entity for its unique identification. Illustrated by a dashed line to represent the lack of dependency.

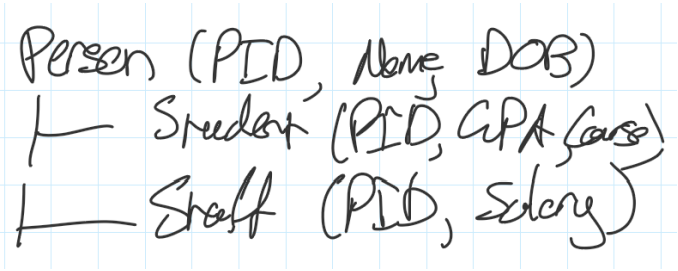
5 ER Digrams - 2

5.1 Associative Entities:

It is best practice to avoid N:M relationships between entities. To do this a third entity is made with 2 1:N relationships. This entity is then an id-dependent on both sides.

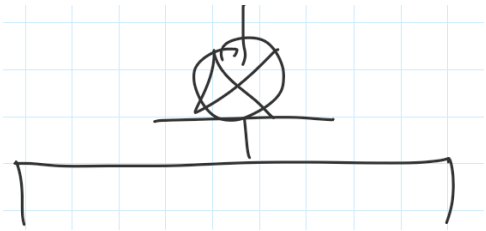
5.2 Subtype:

A type of entity that inherts the attributes and relationships of the more general super-type. Further specifc attributes can be added to the sub-type entities. Sub-type entities reduce reduncy by placing common atrributes in a super-type, whilst also keeping singular attributes in the sub-type.

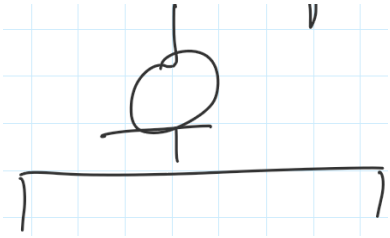


5.3 Notation:

Exclusive is related to atmost one sub-type.



Inclusive is where a super-type can be related to more than 1 sub-type.



Sub-type entities are not suitable when:

- Categories change frequently
- The difference is in a represented value not in its attributes
- Individuals move between categories over time

5.4 Recursive Relationships:

When a FK links too the PK within the same table. For example, an employee is supervised by a manager, who in turn an employee of the company.

6 Normalization

6.1 Data Redundancy:

List and spreadsheets store excess amount of redundant data. This can cause modification anomalies craeting conflicting copies/records in the table. They also have multiple themes per row. Where deletaion anomalies, delete everything within that record, and insertion anomalies, where blank cell are created when data is inserted.

6.2 Relational DBs:

Relational DBs split the spreadsheet into smaller tables that are linked through keys. They are more work to maintain up-front, but solves the problems associated with the spreadsheets.

6.3 Relational Checklists:

All following conditions must be met to make a spreadsheet a relational DB.

Characteristic	Met?
Rows contain data about just one entity	□
Columns contain data about attributes of the entity	□
Single value in each table cell	□
All entries in a column are of the same kind	□
Each column has a unique name	□
Order of columns is unimportant	□
Order of rows is unimportant	□
No duplicate rows	□

6.4 Candidate Keys:

Candidate keys are a unique set of keys that identify a single record. All these keys are capable of becoming a PK.

6.5 Primary Keys:

Notation is used to declare a relation. These include underlining PKs and italicized *FKs*. For exmaple,

TableName(PK, Col1, Col2, *Col3*)

6.6 Boyce-Codd Normal Form:

BCNF is based on a concept that an attribute determines another set of attributes. If nothing determines an attribute, a unique key is added to the system. This is known as a surrogate key.

6.6.1 Referential Integrity:

For tables to satisfy it, every FK value must match a PK value in another table.

6.6.2 NULL Values:

It is better to aviod having NULL values present within DB. This is because the specified value in the row and column is ambiguous.

6.6.3 Functional Dependancy:

Are when an attribute determines all other attribute. The main example for this is when a PK determines all values within a table.

6.7 BCNF:

The BCNF is a form of normalisation:

- Break up tables to remove anomalies
- Stream line data
- Stops replication of data points

7 Database Design

7.1 ER Diagrams to Tables:

Defining the data types present. This determines what will be placed within the column, INT or VARCHAR. Each column also needs to be assigned one specific data type. Further points can be NULL level status, whether a value can be a NULL value or whether a default value should be assigned. In addition the data in the column can also be constrained to a specifc type. E.g. a minimum birth year of 1800.
Each entity within an ERD should have its own table created.

7.2 Constraints:

Unique constraints is similar to a PK, in that each value must be unique to each other within that column. Columns that can have this are usernames or phone numbers. An advantage of this is that the table maintains an idex. Making the sorting of unique data faster.

7.2.1 Checks

Checks force values to be within a user defined range or values. If multiple check clauses are used, the standardised way it to have seperate check statements. A check can also reference multiple columns.

7.3 Referential Integrity:

The FK needs to match PK in the parent table. When something is modified in the parent table, the FK needs to also be modified in the same way. For example, determining if when the FK in parent table is deleted do all rows with FK need to also be deleted.

7.4 Overall Steps:

1. Create a table for each entity
2. Identify the PK
3. Identify data types
4. Verify Normalisation
5. Identify id-dependent/id-independent weak entity relatioships
6. Create relatioships using FKs